

경력기술서

가능성을 기회로 바꾸고, 제품으로 가치를 전하는
메이커 김대정입니다.

김대정

Tech Lead · Backend Engineer ·
실무 4년차

marin.backend@gmail.com github.com/Dae-Jeong marinkim.xyz

경력 요약

제품·Backend·AI

기획자로 시작해 백엔드로 왔고, 아이디어를 제안한 AI 콘텐츠 제품의 FastAPI 백엔드와 AI 생성·평가 시스템을 직접 만들고 핵심 화면은 coding agent로 완성해 월 1천만원 수준의 구독 매출이 발생하는 제품으로 운영합니다.

Python·FastAPI를 중심으로 API·transaction·비동기 worker·AI runtime을 구축하며, 실패를 확인하고 복구할 수 있는 운영 경계를 함께 만듭니다.

MediSolve AI

2025.04 - 현재 · 더데이랩스
프리랜서 2025.02 - 2025.04

Tech Lead · Backend Engineer

AI 활용이 본격화되면서 모두가 메이커로 제품을 만들고 운영하는 팀에서, 제품별 결정·QA·릴리스 운영을 리드하고 Backend·AI application을 직접 구현했습니다. 팀이 같은 기준으로 만들 수 있는 Backend Template과 QA 판정 규칙은 직접 설계했습니다.

Thready · Threads 콘텐츠 제작을 유료 AI 제품으로 전환

아이디어 제안 이후 제품 판단·Backend·AI application·핵심 화면·출시 운영

담당 범위	제품 운영 리드 · Backend/AI 직접 구현 · 핵심 화면은 coding agent로 완성
문제	Threads를 시작하거나 다른 SNS와 병행하기 어려운 사용자가 소재 탐색부터 글 작성·수정·예약·발행까지 반복해서 수행해야 했습니다.
선택	완전 자동 작성보다 탐색·초안·검수·발행을 하나의 제품 흐름으로 묶고, 최종 선택은 사람이 유지하는 방향을 택했습니다.
구현	- FastAPI 제품 API와 별도 AI application·DB를 인증된 HTTP 경계로 분리했습니다. - 소재 탐색·글 생성·수정·예약·발행·품질 라벨링의 핵심 화면을 coding agent로 구현하고 직접 검수·배포했습니다. - typed prompt builder·LLM judge·사람 라벨링을 분리해 생성 결과의 평가 이력을 남겼습니다.
검증	- release·QA·task 기준으로 기능을 운영하고 실제 사용자 흐름에서 회귀를 확인했습니다. - 외부 AI 5xx는 Sentry에서 확인하고 반복 실패 모델만 일시 제외해 다른 모델로 작업을 이어가게 했습니다.

결과	팀과 함께 실제 고객이 구독료를 내고 사용하는 제품으로 출시·운영했습니다. 범위 · 매출은 팀·제품의 결과이며, 제 기여는 아이디어 제안과 prototype 이후 Backend·AI·핵심 화면 구현과 운영 리드입니다.
----	---

Thready · 운영 전환을 위한 FastAPI Backend 재구축

기존 frontend contract를 유지한 병렬 재구축과 단계적 전환

담당 범위	대안 비교·설계·구현·검증·전환 판단 직접 수행
문제	검증에 유리했던 초기 Backend의 domain 의존성과 회귀 위험이 커져 제품 기능을 이어가면서 구조를 바꿔야 했습니다.
선택	부분 수정을 누적하는 안과 Backend만 병렬 재구축하는 안을 비교하고, 서비스 규모와 AI module 확장 시점을 고려해 병렬 재구축을 선택했습니다.
구현	• API·기능 inventory와 응답 비교 기준을 먼저 만들고 기존 frontend contract를 유지했습니다. • domain·repository·transaction 책임을 분리하고 전환 단위를 release로 관리했습니다.
검증	• 동일 기능 비교와 QA acceptance를 통과한 범위만 전환했습니다. • 전환 이후 실제 사용자가 쓰는 Backend를 계속 개발·운영했습니다.
결과	같은 기준의 Jira 집계에서 해결된 QA 이슈 재오픈 비율이 37%에서 11%로, 재발 발생 일평균이 약 94% 줄었고(하루 4.5건에서 0.3건), 전환 이후 실제 사용자가 쓰는 Backend를 계속 운영했습니다.

피부와 운영 제품군 · 주문 후속 작업과 실시간 상담의 실패 경계

주문 worker의 실패 승격과 실시간 상담 runtime의 순서 보장·지연 병목 실측

담당 범위	주문·재고 비동기 Backend 직접 구축 · 실시간 상담 runtime 안정화 공동 수행
문제	주문 transaction 이후의 외부 알림은 별도로 실패할 수 있었고, 실시간 상담에서는 delta 순서와 complete 도착 순서가 달라 문맥이 바뀔 수 있었습니다.
선택	요청 성공과 후속 작업 성공을 분리하고, 복구 가능한 상태와 상담 발화의 순서 기준을 명시했습니다.
구현	• RabbitMQ·TaskIQ worker에 상태·retry·최종 실패·수동 재처리 기준을 연결했습니다. • 상담 세션의 VAD·DELTA·COMPLETE·CORRECTED를 같은 순서 번호로 묶고 중복과 종료 후 재연결을 제어했습니다. • 외부 음성 모델 세션이 발화 중 끊기는 문제를 겹치는 세션 풀과 sequence 병합으로 순서를 보장하며 흡수했고, 모델 교체 뒤 우회 코드를 걷어냈습니다.
검증	• 주문·재고 API와 worker를 분리해 실패 작업을 다시 찾고 처리할 수 있게 했습니다. • VAD 파라미터별 E2E 실측으로 P50 차이가 작고 주 병목이 모델 추론임을 확인해 VAD tuning 대신 DELTA 조기 trigger에 집중했습니다. • 종료 뒤 재연결되던 경합 경로를 재현해 13개 focused regression으로 고정했고, 실시간 상담 회귀 테스트에서 발화 문맥과 완료 순서를 대조했습니다.

결과	API 성공 뒤 실패를 숨기지 않고 운영자가 복구 지점을 확인할 수 있는 흐름을 만들었습니다. 범위 · 주문 worker는 주도, 실시간 상담은 공동 주 기여이며 두 runtime을 하나의 시스템처럼 합치지 않습니다.
----	--

Backend Template · Coding Agent가 따를 FastAPI 기본값

AI 활용이 본격화되면서 모두가 메이커로 제품을 만들고 운영하는 팀의 공통 기준

담당 범위	Template 직접 설계·구축 · 팀 피드백 반영 · 도입 지원
문제	팀이 coding agent로 기능을 만들기 시작하면서 백엔드 경험이 적은 담당자도 구현에 참여했습니다. QA와 운영 준비 단계에서 사용량이나 동시 요청 조건에 따라 구조적인 문제가 자주 드러났고, 그때마다 백엔드 엔지니어의 리소스가 원인 파악과 보완에 쓰였습니다.
선택	기능마다 완벽한 구조를 요구하기보다, 반복 실패를 막는 session·transaction·API contract를 기본값으로 제공했습니다. 일반 기능은 MVC, tool 확장이 많은 Agent 기능은 Hexagonal 경계를 권장했습니다.
구현	• @transactional decorator와 ContextVar 기반 AsyncSession resolve를 구현했습니다. • 폴더·책임·API 응답·검증 규칙과 Agent context를 Template에 함께 넣었습니다.
검증	• 정기 개발 미팅에서 구조화 비용과 패턴 선택 기준을 피드백 받아 구현에 반영했습니다. • 신규 사내 프로그램은 Full Template으로 시작했고 기존 제품은 session 관리부터 순차 도입했습니다.
결과	full template으로 시작한 신규 프로그램의 STG QA에서 같은 유형의 session·connection 문제가 재관측되지 않았고, 기획·QA·디자인 담당자가 직접 구현하는 동안 백엔드는 결과 피드백과 배포 지원으로 개입을 줄였습니다. 범위 · Template의 설계·구축은 직접 했고 각 기능 구현은 담당자의 몫이었으며, 회사 AX 전체 구현과는 구분합니다.

제품 개발 체계 · 판단의 정확이 유지되는 기록 구조와 evidence로 닫는 QA 판정 규칙

제품별 결정·명세·작업·QA·릴리스 운영 리드, QA 팀과의 하네스화, 회사 AX 구조 설계 참여

담당 범위	제품 단위 운영 리드 · QA 판정 규칙 설계 · 회사 AX 설계 참여
문제	제품 간 맥락 공유가 사람에 의존해 요구·판단을 재확인하는 비용이 컸고 담당 교체·병행 시 실행이 지연되는 병목이 있었습니다. QA는 실행이 성공하면 통과로 보는 경우가 있어 AI 기능의 품질 판정이 흐려졌습니다.
선택	결정·명세·작업·릴리스 근거를 실행 원장으로 축적해 사람과 AI가 같은 맥락을 읽게 하되, 우선순위·승인·릴리스처럼 판단이 필요한 단계는 사람이 확정하는 경계를 지켰습니다. QA는 요구사항을 REQ로 쪼개 evidence로 닫는 판정 규칙을 세우고 실행 성공과 품질 통과를 분리했습니다.
구현	· 제품별 pipeline registry에 결정·명세·작업 기록을 실행 원장으로 적용하고 BE·FE·QA 담당 lane과 QA 승인을 release gate에 연결해 release note 생성을 자동화했습니다. · 전 제품 공통 Quality Evidence Harness를 QA 팀원의 서포트를 받아 작성했습니다. REQ별 evidence plan(FE·DB·Log·Network·AI quality), PASS/FAIL/UNKNOWN 판정, 반복 이슈의 자동화 승격, LLM judge 단독 승인 금지가 규칙입니다. · 이 판정 layer를 QA 팀이 운영하는 AI QA 에이전트 파이프라인(시나리오→TC→자동화→버그 등록)에 연결하고 백그라운드 작업의 고장 모드 카탈로그와 fault-injection QA를 설계로 제안했습니다.
검증	· 팀 주 1회 agent 활용 회고로 무엇을 만들지와 구현 방법을 함께 정했고 Slack에서 접수한 이슈를 사내 시스템에 남겨 해결 상태를 추적하는 흐름을 제안했습니다.

결과	담당자가 바뀌어도 기록에서 업무를 이어가 인수인계·맥락 복원 비용이 낮아졌고 유사 기능에서 기존 판단을 재사용해 lead time 단축에 기여했습니다. 이 구조를 회의·의사결정·업무 배정·승인으로 넓히는 회사 AX 설계에 참여했습니다. 범위 · 전사 문서 규칙 정의와 QA AI 에이전트 플러그인 구현은 다른 담당자가 주도했습니다. 제 범위는 제품 단위 적용·운영 리드, 공통 QA 판정 규칙 설계, AI QA 팀 운영 참여입니다. lead time·QA 시간의 수치는 측정하지 않았습니다.
----	--

Thready · 제품 원장과 AI 실행 상태의 분리·Outbox 전달

AI 실행부의 독립 application·DB 분리, STG 실테이터 이관, 전달 보장·먹등 소비·정합 검증

담당 범위	경계 설계·구현·migration·검증 직접 수행
문제	제품 정책·원장과 AI 생성 lifecycle이 한 Backend·DB에 있어 AI 확장과 장애가 원장에 결합됐고, 분리 뒤에는 전달 유실과 역순 도착이 최신 상태를 덮을 수 있었습니다.
선택	DB를 공유하지 않고 독립 FastAPI application·DB로 분리해 인증된 HTTP 계약으로만 연결하고, 전달은 원장 변경과 같은 transaction에 기록하는 Transactional Outbox로 분리했습니다.
구현	• lease 기반 claim·attempt token·retry·delivery version fence·먹등 consumer·terminal failure 보존을 구현했습니다. • STG 생성 이력·품질 기록·실행 추적을 parent→child 순서로 streaming copy했고 영구 cross-DB link는 쓰지 않았습니다. • 생성 원장은 version CAS로 다중 worker 경합을 중재하고 상태 전이 규칙을 entity에 복원했으며, quota는 예약과 admission gate로 동시 요청의 초과 실행을 막았습니다.
검증	• local rehearsal·행 수·MD5 fingerprint·FK orphan 0건으로 이관 정합성을 확인했습니다. • workflow·health가 성공해도 생성이 실패한 사례를 계기로 배포 성공과 기능 동작을 분리한 post-deploy 생성 API E2E gate를 세웠습니다.

결과	독립 AI application·DB를 STG·Prod에서 운영 중이며 지연·중복·역순 전달이 최신 원장 상태를 덮지 않는 복구 경계를 확보했습니다. 범위 · Prod migration 완료나 무중단 전환은 주장하지 않습니다.
----	--

여러 피부과의 운영·예약 Backend와 지점 접근 경계

외부 피부과 여러 곳의 홈페이지·관리·예약을 지원하는 multi-tenant backend monorepo (진행 중)

담당 범위	Backend architecture·Admin/Homepage API 구축 주도 · Terraform IaC 전담
문제	여러 피부과의 운영·예약을 한 Backend가 지원해야 했고, client가 보내는 header가 tenant 접근 범위를 결정하던 구조가 격리의 failure mode였습니다.
선택	Admin·Homepage API를 gateway 뒤 독립 모듈로 나누고, working branch를 server auth state로 소유해 권한 검증 전용 API로만 전환하되 Homepage API의 기존 계약은 유지했습니다.
구현	• Router–Service–Repository–Model과 DI, Generic Repository, multi-tenancy·soft delete 자동 필터를 구성했습니다. • 본사 미선택 409·권한 밖 403을 구분하고 Ruff·Pyright·pre-commit 정적 검증과 API·DB 설계 가이드를 구축했습니다.
검증	• shared auth middleware와 Homepage API 계약을 유지한 채 Admin API 범위에서 접근 경계를 전환했습니다.

결과 제품은 예약률 개선으로 고객사 매출 성과에 기여했고, tenant 접근 범위가 server 상태로 결정되는 경계로 전환하고 있습니다.

범위 · 2026-08 기준 진행 중이며 예약률·매출은 제품·팀 outcome입니다.

Memento AI

2024.10 - 2025.01

Backend Engineer

예약·결제·고객 알림 Backend를 구현했습니다. 회사 폐업으로 경력이 종료되었습니다.

예약·결제·고객 알림 Backend

FastAPI·SQLAlchemy·MySQL 기반 예약 서비스

담당 범위	예약 API·외부 결제·알림 작업 구현
문제	외부 결제 결과와 내부 예약 자산 변경이 한 번에 성공하지 않을 수 있었습니다.
선택	결제 취소·환불과 내부 상태 변경 시점을 분리해 실패 지점을 확인할 수 있게 했습니다.
구현	- 예약 API와 외부 결제를 연동하고 결제 상태에 따라 내부 자산 변경을 제어했습니다. - 다국어 알림의 즉시·예약 발송과 작업 취소·재등록·발송 이력을 구현했습니다.
검증	결제·예약·알림 상태를 분리해 실패 시 재처리 범위를 확인했습니다.
결과	예약과 결제, 후속 알림의 책임을 나눠 운영 가능한 상태 흐름을 만들었습니다.

STUDIO LAB

2021.12 - 2024.01

Vision AI Engineer → Product Manager → Backend Engineer

AI 모델에서 시작해 제품 기획과 Backend까지 역할을 넓히며 생성형 AI 커머스 제품을 만들었습니다.

SellerCanvas · 생성형 AI 커머스 제품

prototype에서 v1.0과 외부 기업 PoC까지 발전

담당 범위	PM 중심 · AI/Backend 경험을 바탕으로 제품 범위와 출시 우선순위 결정
문제	AI 기능을 데모가 아니라 실제 업무 흐름에서 쓸 수 있는 제품으로 구체화해야 했습니다.
선택	사용자 흐름과 기능 범위를 먼저 정하고 모델·Backend·화면을 제품 단위로 연결했습니다.
구현	프로토타입의 기능 범위를 정리하고 v1.0 제품과 기업 PoC로 확장했습니다.
검증	사용 시나리오와 PoC 요구를 기준으로 기능 범위와 우선순위를 조정했습니다.

결과 CES 2024 Best of Innovation AI 부문 대상 제품에 참여했습니다.

범위 · 회사·팀 제품 성과이며 개인 단독 수상으로 표현하지 않습니다.

TellingMe · 개인 프로젝트

2024.01 - 2024.12

Backend Lead · Infra

Memento AI 재직 기간과 일부 겹치는 개인 프로젝트이며 정규 경력과 분리해 기록합니다.

TellingMe · 개인 프로젝트 Backend

Spring Boot Backend와 AWS 배포

담당 범위	Backend Lead · Infra
문제	개인 프로젝트의 API와 배포 환경을 한 흐름으로 운영해야 했습니다.
선택	Spring Boot Backend와 AWS 기반 배포 구성을 함께 맡았습니다.
구현	Backend API 개발과 서비스 배포 환경을 구성했습니다.
검증	개인 프로젝트의 운영 범위에서 배포와 기본 운영을 확인했습니다.

결과 제품 Backend와 배포를 함께 맡은 개인 프로젝트 경험을 확보했습니다.

범위 · 정규 경력이 아니며 개인 프로젝트로만 표기합니다.

아이즈슬

2020.08 – 2021.06

Vision AI Engineer · Intern

Vision AI 모델과 데이터 파이프라인을 제품 기능으로 연결하는 개발·검증에 참여했습니다.

기술

Backend

Python · FastAPI · PostgreSQL · MySQL · Redis · SQLAlchemy

Async / Realtime

RabbitMQ · TaskIQ · WebSocket · SSE · STT

AI Product

LLM integration · typed prompt · structured output · evaluation · labeling

Delivery

Docker · GitHub Actions · Terraform · Azure · AWS · Kubernetes(kubeadm · Calico · MetalLB · ArgoCD)
